Claims Orchestration on Temporal

Entity Workflow: one workflow per claim

1. Summary

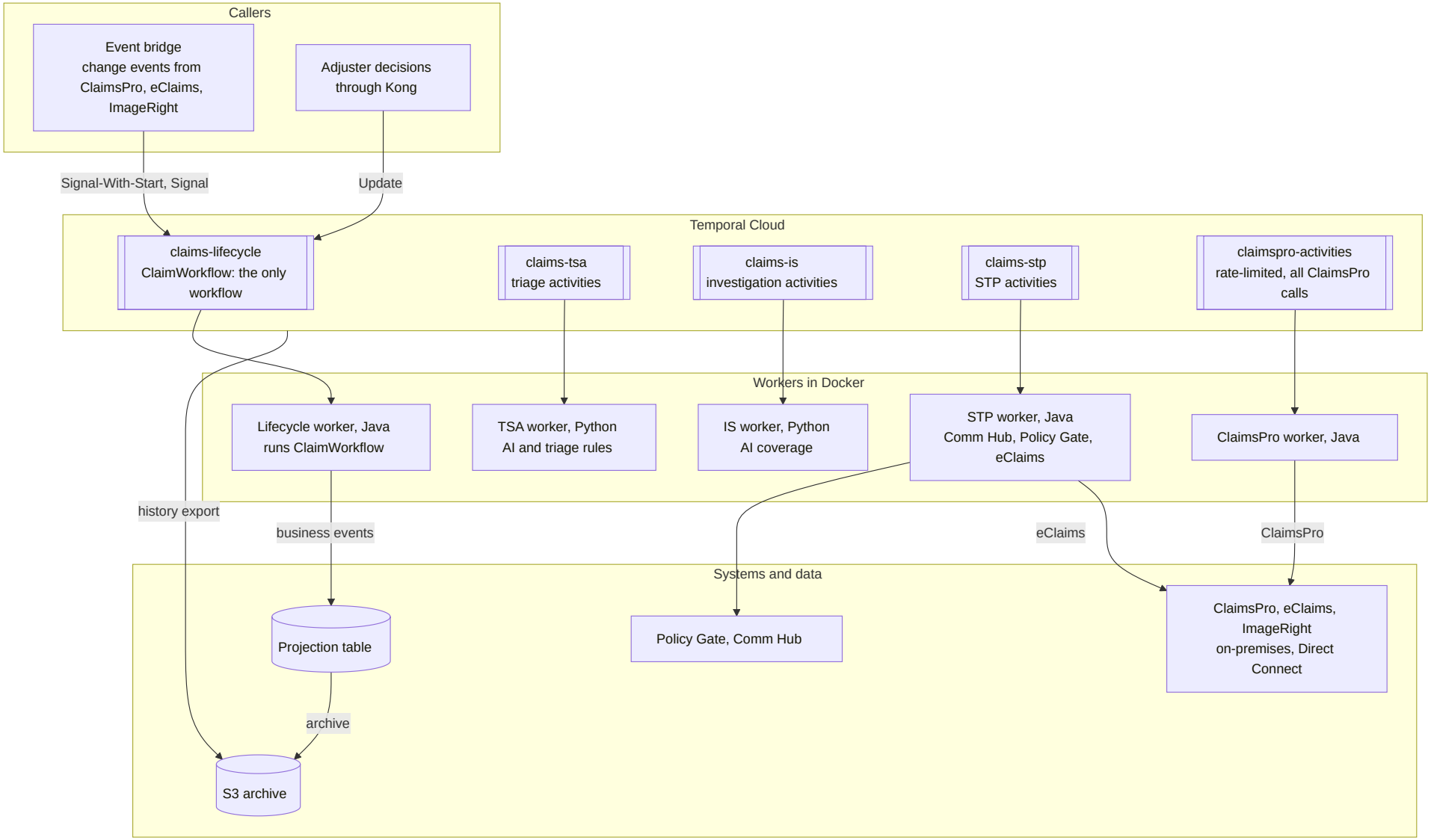
Each claim is one long-running **ClaimWorkflow**: the standard Temporal entity-workflow pattern, one workflow per business entity for its whole life. Triage, investigation and STP are modules inside it, not separate workflows. Their work runs as activities on each track's own task queue and worker deployment: the Python team's AI and triage-rule activities on claims-tsa and claims-is, the Java STP activities on claims-stp, every ClaimsPro call on the rate-limited claimspro-activities queue. Events and every human decision arrive at the same workflow ID, `claim:{claimId}`. Payment runs in a region of the workflow that is never cancelled. Application data, S3, contracts and security are the same production baseline as Parent–Child Orchestration.

2. Why one workflow

Is a child workflow needed at each stage? Temporal's guidance is that child workflows are not a way to organise code; they are justified by a separate service or worker fleet that owns the workflow, a need to partition a very large history, or a one-to-one mapping to a separate resource. In this claims platform the tracks need separate *workers* (Python AI, Java STP) and separate *deployments*, which activities on their own task queues already give. The orchestration itself (stage, events, decisions, re-triage) is one coherent state machine for one claim, so it lives in one workflow.

Property	What one workflow gives
One history per claim	Everything that happened to a claim is in one execution (and its Continue-As-New chain)
One front door	Events, decisions and queries all go to `claim:{claimId}`; nothing is forwarded or relayed
One place for cross-track rules	Re-triage limits, material change, stage version, payment authority live together
Fewer actions	No child workflow starts (two actions each) and no forwarded signals
Independent deployment where it matters	Activity workers (Python AI, Java STP) deploy without workflow versioning concerns, as long as the activity contract holds

3. Temporal architecture

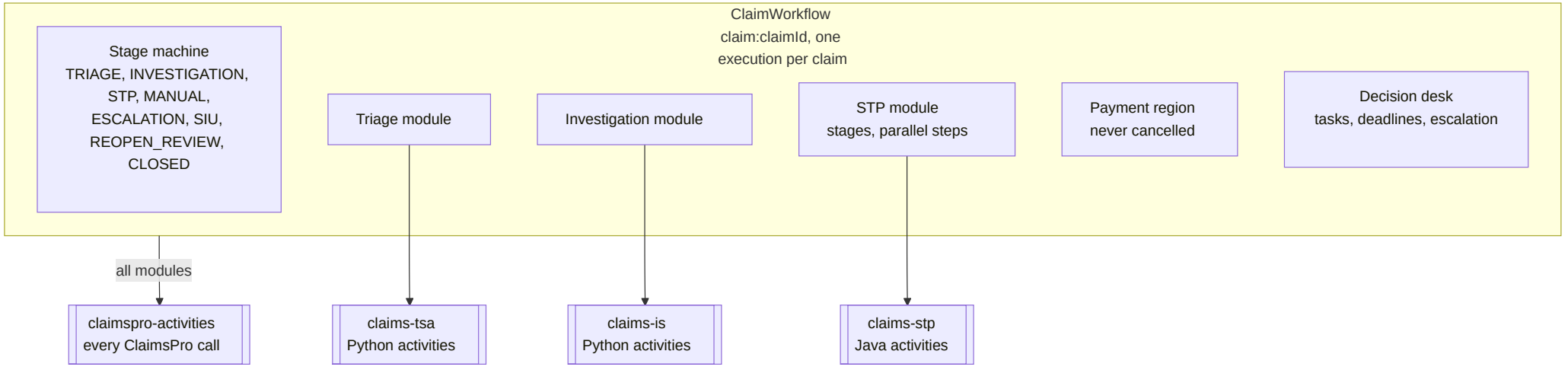


Task queue	Runs	Worker deployment
claims-lifecycle	ClaimWorkflow (the only claim workflow), ArchiveWorkflow; projection, policy and archive activities	Lifecycle worker, Java (Auto-Upgrade)

Task queue	Runs	Worker deployment
claims-tsa	Ai.assessRisk, Triage.applyRules	TSA worker, Python (activities only)
claims-is	Ai.analyzeCoverage	IS worker, Python (activities only)
claims-stp	Comm Hub, Policy Gate, eClaims and flow activities	STP worker, Java (activities only)
claimspro-activities	Every ClaimsPro call; rate limit on the queue	ClaimsPro worker, Java

Connections and deployment are as in Parent–Child Orchestration: outbound gRPC with mTLS through PrivateLink, Direct Connect to on-premises systems, Kong for inbound calls, one EKS deployment per queue scaled on its latency.

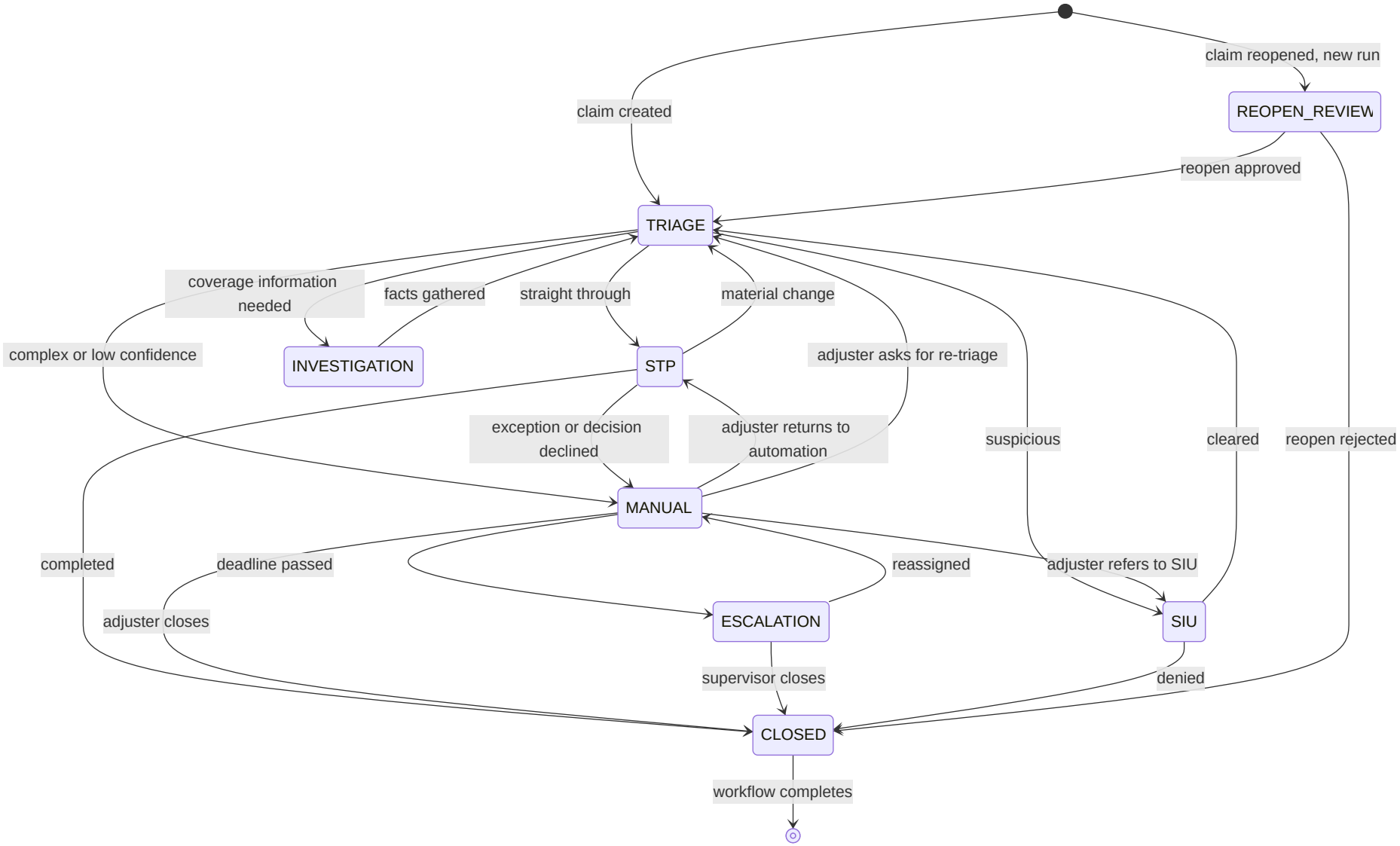
4. Inside the claim workflow



Module	Responsibility
Stage machine	The claim's stage and stage version; routing between stages; re-triage limit; Continue-As-New between stages
Triage module	Read claim, AI risk, triage rules, triage review when confidence is low; returns a route
Investigation module	AI coverage analysis; waits for documents or an IS reviewer; returns to triage

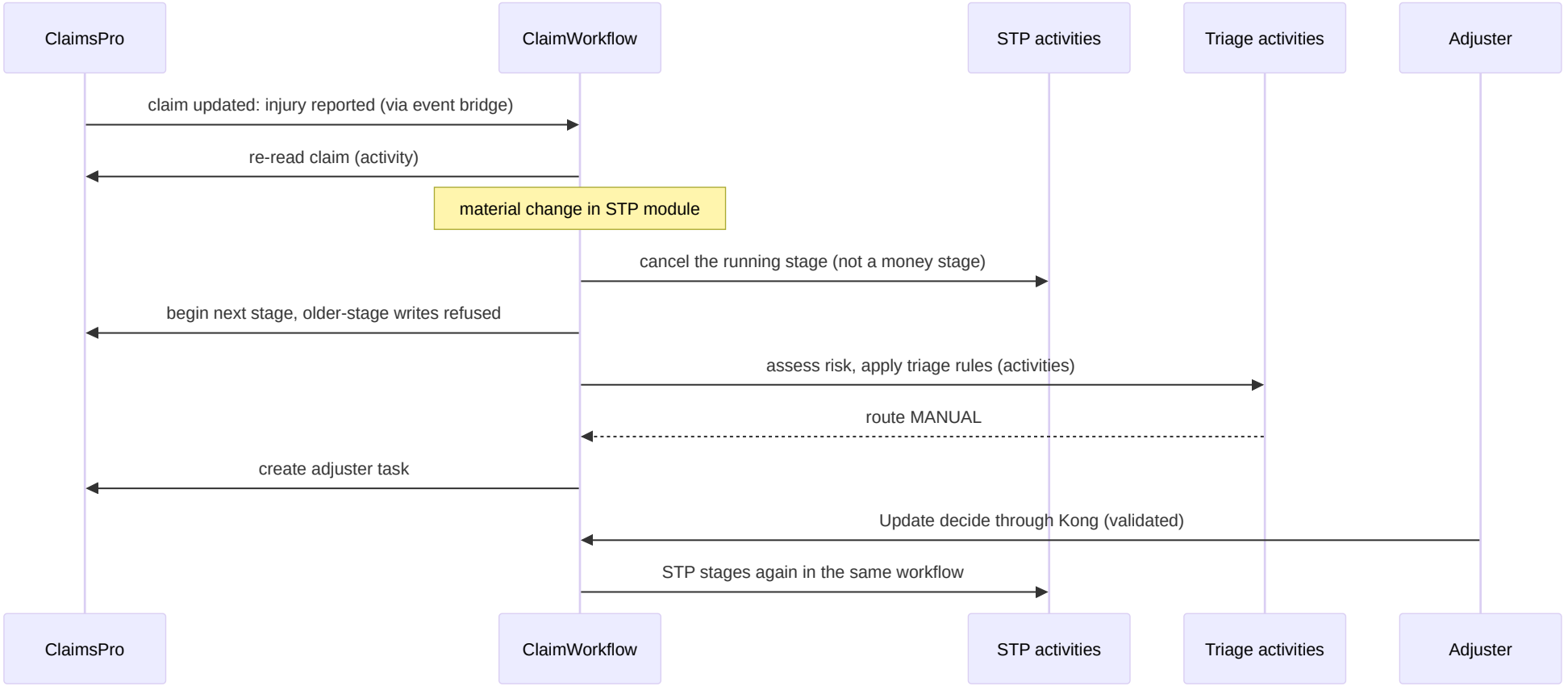
Module	Responsibility
STP module	Runs the claim's flow definition: stages in order, steps of a stage in parallel; vendor dispatch, appraisal, estimate, reviews
Payment region	Approval above authority, create (payment key looked up first), close; run outside every cancellation scope
Decision desk	Every human decision: task in ClaimsPro, deadline, escalation, validated Update

Claim stages



Every stage change increments the stage version in the projection table; writes from an older stage are refused. The workflow completes when the claim closes; a reopen later starts a new run with the same workflow ID and continues the stage numbering. It continues as new between stages when the history is large, carrying its state, so one claim's history never approaches Temporal's limits.

A claim change during STP



Events are deduplicated and handled where they are needed, in the same workflow. On a claim change the workflow re-reads ClaimsPro and applies the material-change rule. A material change cancels the running STP stage at once (its activities are cancelled) and moves the claim to triage; the payment region is never cancelled and finishes first. Once money has moved, the change is recorded and STP closes the claim; a reopen brings it to an adjuster.

5. Human decisions

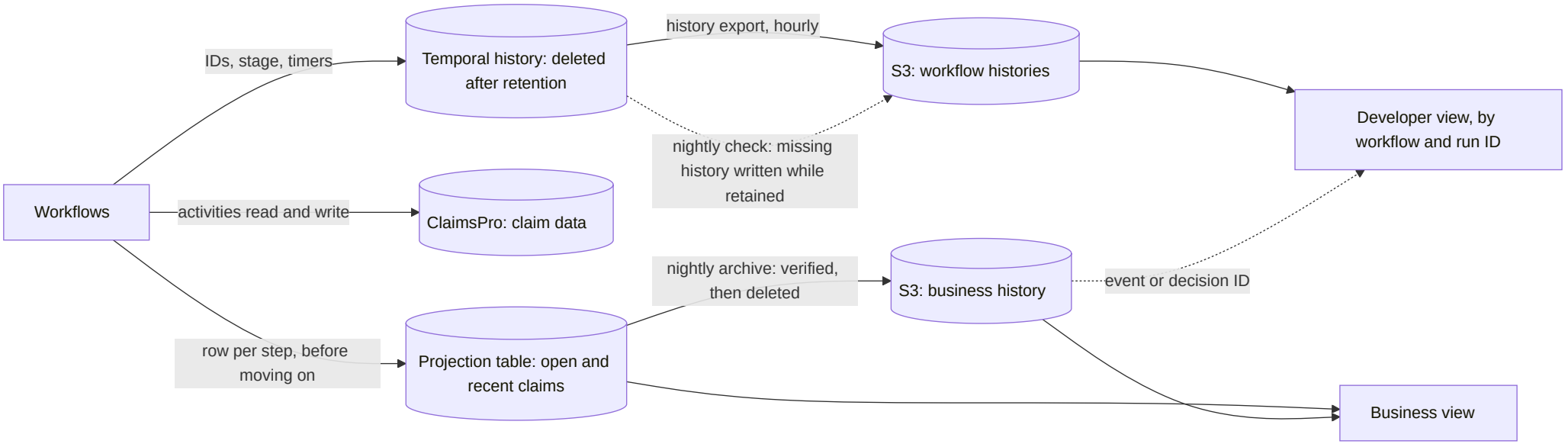
Every decision follows one lifecycle in the workflow: adjuster task created in ClaimsPro with the claim ID and decision ID; deadline timer; escalation through the roles in the authority matrix; the adjuster's decision sent by the service behind Kong as Update `decide` to `claim:{claimId}`, decision ID as Update ID; validation before anything is

recorded (task open, choice offered, role, authority, requester is not the approver, reason where required); task closed, row recorded; open tasks withdrawn when the claim moves on.

Decision	Module	Role, then escalation
Triage review (low AI confidence)	Triage	Triage adjuster, triage supervisor
Manual handling; escalation; SIU; reopen review	Stage machine	Adjuster, supervisor, SIU group
Coverage information	Investigation	IS reviewer, supervisor
Customer not reached; no vendor; estimate review	STP	Adjuster, appraiser, supervisor
Payment above authority (four eyes, reason to reject)	Payment region	Approver with authority, claims manager
A step still failing after its retry window	Any	Claims operations

6. Data outside Temporal and S3

Option for S3	Where it applies	Use here
Temporal Archival (S3 archiver)	Self-hosted Temporal only; experimental; not supported in Docker	Not available on Temporal Cloud
Temporal Cloud Workflow History Export	Every closed history to the client's S3 bucket hourly, protobuf, at least once; IAM role and KMS key	Used: the developer record
Projection archive	Nightly workflow: business rows of claims whose every run has closed go to S3, verified, then exactly those rows deleted	Used: the business record



As in Parent–Child Orchestration: Temporal holds IDs, stage and timers; ClaimsPro keeps claim data; the projection table keeps business history, written before the workflow moves on; Temporal Cloud has no Archival, so its Workflow History Export (hourly, protobuf, at least once) is the developer record in S3, with a nightly check that copies any missing history while it is still retained; the projection table is archived nightly to S3, verified before deletion. One claim, one workflow ID: the business row and the exported history join on it directly.

7. What we give to Temporal

Temporal Cloud runs the orchestration only. It receives IDs, stage, timers, decisions and encrypted payloads; claim data stays in ClaimsPro and business history in the projection table. This is the complete list of what is set up in Temporal Cloud for each environment; everything else runs in the client's AWS account.

Temporal Cloud account and namespace

Item	Production setting
Namespaces	One per environment (development, test, production), in the AWS region of the EKS workers
Connectivity	Workers and the event bridge connect outbound only, gRPC over AWS PrivateLink; no inbound path into the client network
Authentication	mTLS client certificates from the client's CA, one per environment, rotated before expiry

Item	Production setting
People	SSO; namespace roles: administrators (platform team), read for developers and support; no write access to running claims
Retention	30 days after a workflow closes (open claims are never deleted)
Workflow History Export	Every closed history to the client's S3 bucket, hourly, protobuf; IAM role with write access to that bucket only; client KMS key
Codec server	Decrypts payloads in the Temporal UI for authorised users; runs in the client's account behind SSO and holds the payload keys; Temporal never sees the keys
Capacity	Actions per second sized from the catastrophe peak (500 by default; increase requested ahead of the season)
Replication	Three availability zones by default; multi-region replication optional (recovery point under one minute, recovery time about 20 minutes)
Metrics	Temporal Cloud metrics endpoint scraped into the client's dashboards, with the SDK metrics from every worker

What runs in the namespace

Temporal object	Values
Workflow types	ClaimWorkflow, ArchiveWorkflow
Task queues	claims-lifecycle (the claim workflow), claims-tsa, claims-is, claims-stp (activities only), claimspro-activities with a server-side rate limit at ClaimsPro's agreed rate
Worker deployments (Worker Versioning)	claims-lifecycle, claims-tsa, claims-is, claims-stp, claims-claimspro; the claim workflow Auto-Upgrade with patching; activity-only deployments release freely; a new build is ramped, then made current
Search attributes (Keyword)	ClaimId, Stage, OpenDecision: IDs and codes for finding a claim and its open decisions
Signals and Updates	Signal claimEvent (event bridge), Signal cancelPayment, Update decide with validator (service behind Kong)
Schedule	claims-nightly-archive: ArchiveWorkflow at 02:00 daily; archives claims closed more than 30 days and checks the history export
Payload encryption	AES-256-GCM on every payload and failure message, same codec in Java and Python; keys from the secrets manager, rotated with the previous key kept for decoding

Never given to Temporal: personal data in workflow IDs, search attributes or logs (they hold IDs and codes only), claim documents, payment details beyond IDs and amounts, database credentials or system endpoints.

8. Running in production

Area	Entity Workflow
Versioning	One long-lived workflow type on Auto-Upgrade: every change is patched and replay-tested against recorded histories. Activity workers release independently; an activity change needs no workflow versioning
History size	All tracks in one history: Continue-As-New between stages when suggested, well below the 10,240-event warning
Throughput	Workflow tasks only on claims-lifecycle; activity queues scale per track; ClaimsPro protected by the queue rate limit; capacity sized for the catastrophe peak
Cost	No child starts and no forwarded signals: about 55 actions per straight-through claim, against about 67 for Parent–Child Orchestration
Failure isolation	A failing activity retries within its window, then opens an operations task; a workflow code defect affects every track of the claim, so replay tests gate every release
Security, observability	As Parent–Child Orchestration: mTLS, PrivateLink, shared payload codec, IDs only in search attributes, key rotation, no mTLS without a payload key; ELK logs, LAMP for AI, Temporal metrics

9. Parent–Child Orchestration and Entity Workflow compared

	Parent–Child Orchestration	Entity Workflow
Temporal pattern	Parent workflow with child workflows	Entity workflow with activities
Workflows per straight-through claim	4 (claim, triage, STP, payment)	1
Who writes orchestration code	Each track team writes its own workflow (Python TSA and IS, Java STP)	The lifecycle team (Java); track teams write activities
Events	Parent forwards to the active child	Handled in place
Human decisions	Update on the waiting workflow; task carries its workflow ID	Update always on claim:{claimId}

	Parent–Child Orchestration	Entity Workflow
Versioning	Parent Auto-Upgrade, children Pinned per track	One workflow Auto-Upgrade with patching; activity workers free
History	Split across child executions	One history per claim, more Continue-As-New
Actions per straight-through claim (measured)	About 67	About 55
Measured surge (200 claims, 20 per second, local)	All closed in 85 s	All closed in 59 s
Blast radius of a workflow bug	One track	The whole claim workflow
Choose when	Track teams must own and release their workflows independently	One team owns the claim process; simplicity, cost and one view of the claim matter most

10. Pros and cons

Pros	Cons and how they are handled
Simplest model: one claim, one workflow, one ID for everything	One workflow type carries all tracks: kept in modules, reviewed and replay-tested as one
No event forwarding or decision relay; nothing to keep in sync between workflows	Python teams write activities, not workflows: their AI logic is the activity anyway
Fewest actions and executions per claim	History grows with every track: Continue-As-New between stages
Activity workers deploy freely	Every workflow change must be patched, as the claim workflow lives for the whole claim
Payment and re-triage rules in one place	A workflow defect stops the whole claim, not one track: replay tests and ramped rollout

11. Claim scenarios verified end to end

Scenario	What it proves
Straight through	Full STP; a lost reply from ClaimsPro still gives exactly one payment

Scenario	What it proves
Injury during STP	The running estimate stage is cancelled at once; re-triage; an adjuster returns the claim to STP
Coverage investigation	Investigation waits; documents arrive; re-triage; STP
Fraud	SIU denial without a reason is rejected; with a reason it is accepted
Large payment	Estimate review; approval rejects the requester and an approver without authority
Catastrophe, lost event	Coverage, reserve and appraisal in parallel; the lost estimate event is recovered by the timer re-read
Customer not reached	An adjuster asks for another contact attempt
Low AI confidence	A triage adjuster chooses the route
Closed in ClaimsPro, reopened	The running module is cancelled and its task withdrawn; the reopen is a new run
Payment cancelled in ClaimsPro	A cancel during approval stops the payment; an adjuster closes the claim with a reason